

CS7GV5 Real-Time Animation: Motion Editing

Priyansh Nayak

April 20, 2026

Demo link: <https://youtu.be/IukL8HaHxQ8>

Overview

This project is built around three motion-editing demos in Unreal Engine, each covering a different part of the animation pipeline. The first focuses on retargeting the same walking clip onto three MetaHumans with different proportions, the second develops a two-character Rock-Paper-Scissors interaction system, and the third looks at facial performance transfer using Live Link and MetaHuman Performance assets with clips from the RAVDESS dataset. Even though the demos are quite different from each other, they all point back to the same basic issue: source animation is rarely strong enough to use as-is.

1 Retargeting and Character Diversity

For the first demo, I used a single walking motion capture clip and transferred it onto three different MetaHumans with noticeably different proportions. The three characters were my own avatar, built as a roughly six-foot, slightly chubby male and presented using Epic Games' MetaHuman Techwear Outfit [8], together with two alternative MetaHumans: a short female character using the Casual Hoodie & Jeans outfit by polycornStudio [14], and an older male character using the Poor Man Outfit by NOETA Marketstore [13]. Since the source clip came from my own mocap recording, it was naturally best suited to my own avatar, which made it a useful baseline for seeing what changed when the same motion was applied to characters with very different body types. That also pushed this away from being just a transfer test and toward what had to happen afterwards, because what looks acceptable on one character started to feel too light, too stretched, or badly grounded on another.

1.1 Retargeting Workflow

The initial workflow followed Unreal Engine's standard retargeting pipeline [5]. The source mocap skeleton was imported, IK rigs were generated for both source and target characters, and an IK Retargeter was created to transfer the motion. At a basic level this worked, but the first results exposed several problems:

- root misalignment that caused the character to disappear or collapse visually,
- pelvis height errors that pushed the body too low,
- foot penetration into the ground,
- arm offsets caused by pose mismatch between source and target skeletons,
- and instability introduced by IK goals and solver settings that did not suit the source rig.

These issues made it clear that the default setup was not robust enough for this dataset, so a debugging pass was needed before the retargeting results became usable. IK goals that referenced incompatible bones were removed, the Full Body IK solver was disabled in the retargeting configuration, and the retarget root was changed to the pelvis or hips. I also found that Unreal’s post-process animation logic could interfere with testing, so that system had to be disabled temporarily while verifying the raw retarget result.

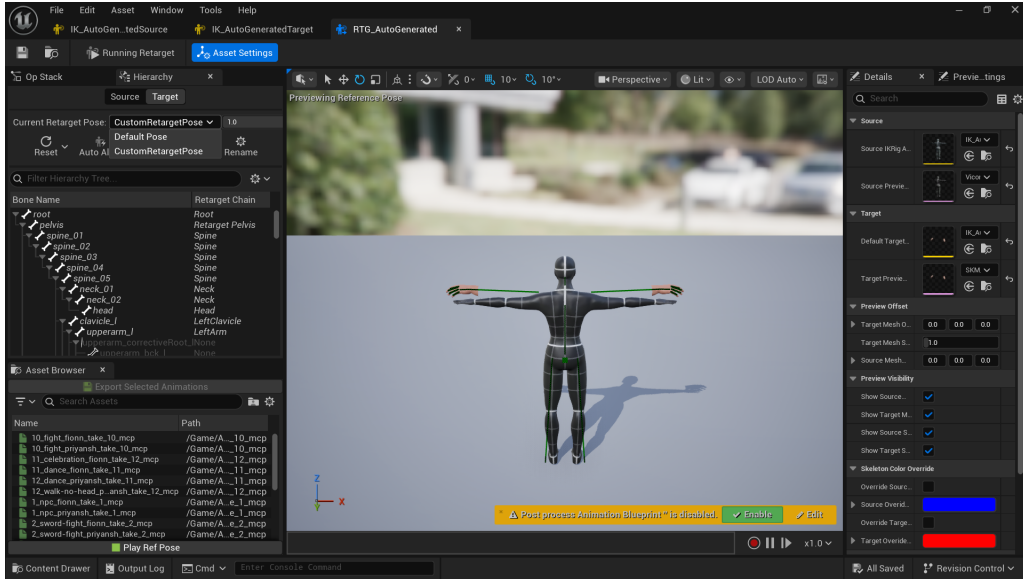


Figure 1: IK Retargeter pose transfer from mocap to MetaHumans.

1.2 Per-Character Adaptation

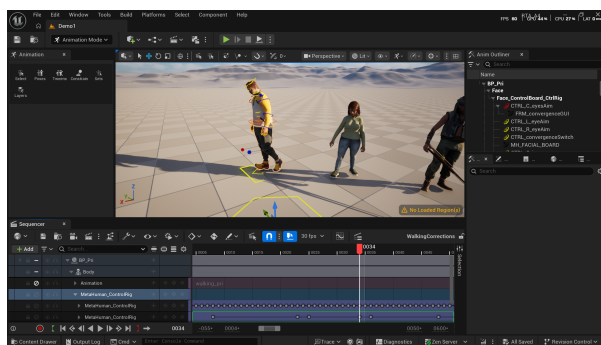
My initial assumption had been that I could retarget the animation once and then reuse the result across all characters. In practice, this produced uneven quality because the same source walk was interpreted very differently depending on the target character’s build. My own avatar needed the least correction overall, since the mocap came from my own performance, while the shorter female character and the older male character each revealed much clearer exaggerations in posture, stride length, and grounding.

Because of this, I changed the workflow and created a separate retargeting pass for each MetaHuman rather than relying on a single shared export. This allowed me to make small but important adjustments per character, including pelvis height correction, posture changes,

root alignment tweaks, and later playback adjustments. In practice, my own avatar stayed closest to the original clip and therefore kept its original speed, the female character was made to feel lighter and more energetic in the stride but ended up only very slightly slower because of the shorter step length, and the older male character was clearly the slowest, with stiffer steps and a more obviously adjusted posture. Although this increased workload, it gave far more control and produced noticeably cleaner results.

1.3 Sequencer and Control Rig Editing

The retargeted clips were then taken into Sequencer and baked to Control Rig for cleanup using the MetaHuman Control Rig workflow [1]. The main corrections during this stage were foot placement adjustments to reduce ground penetration and sliding, fixes to the hips, spine, and shoulder rotations, visible jitter cleanup, posture refinement for each character, and smoothing of awkward moments in the stride cycle. These edits improved grounding and made the differences between the characters more apparent instead of all three looking like simple copies of the same clip.



(a) Posture cleanup.



(b) Stride distance and grounding.

Figure 2: Sequencer keyframe adjustments for posture, stride distance, and speed.

To strengthen the distinction between characters, I also adjusted play rate and stance slightly per character. My own avatar stayed closest to the original clip, the shorter female character was shaped to feel lighter and more energetic even though the shorter step length made her read marginally slower overall, and the older male character was pushed toward a slower and stiffer result.

1.4 Critical Evaluation

The most useful thing this demo showed is that retargeting is not always simple. The transfer itself worked, but the raw result still was not strong enough to present as finished motion, and the biggest lesson here was how strongly body proportions affect perception even when the underlying source clip stays exactly the same. It also shows that once proportions start changing, the motion also needs to be adapted, and the combination of per-character retargeting and Control Rig cleanup made a much bigger difference than the transfer alone.

The weakness is that the workflow is still very manual, because different characters produce different grounding and pose issues and the amount of cleanup scales up quickly once more than one body type is involved. Even after cleanup, there can still be small problems like slight sliding or imperfect foot contact.

Overall, I think the demo works because it shows both the usefulness and the limits of retargeting honestly. It makes the case that believable reused motion usually comes from retargeting plus editing, not retargeting on its own.

2 Two-Character Interaction

The second demo was built around a Rock-Paper-Scissors interaction between a player-controlled MetaHuman and an NPC. The structure was simple: the player has to choose an action from the UI, the system has to resolve the result, and both characters then have to respond in a way that feels synchronized, readable, and spatially correct. Most of the clips in the set came from my own mocap recording, which made them particularly well suited to translation onto my MetaHuman avatar and gave the player side of the system a more natural baseline before cleanup.

2.1 System Structure, Logic, and Clip Setup

The whole interaction is coordinated through a central manager blueprint. That manager receives the player’s move, generates the NPC move, resolves the outcome, informs the two MetaHumans which move and result to play, updates score values, and prevents rounds from overlapping. I kept the gameplay logic there because it made the rest of the system easier to reason about: the player and NPC only needed to worry about animation playback and local state, while the manager handled the round itself.

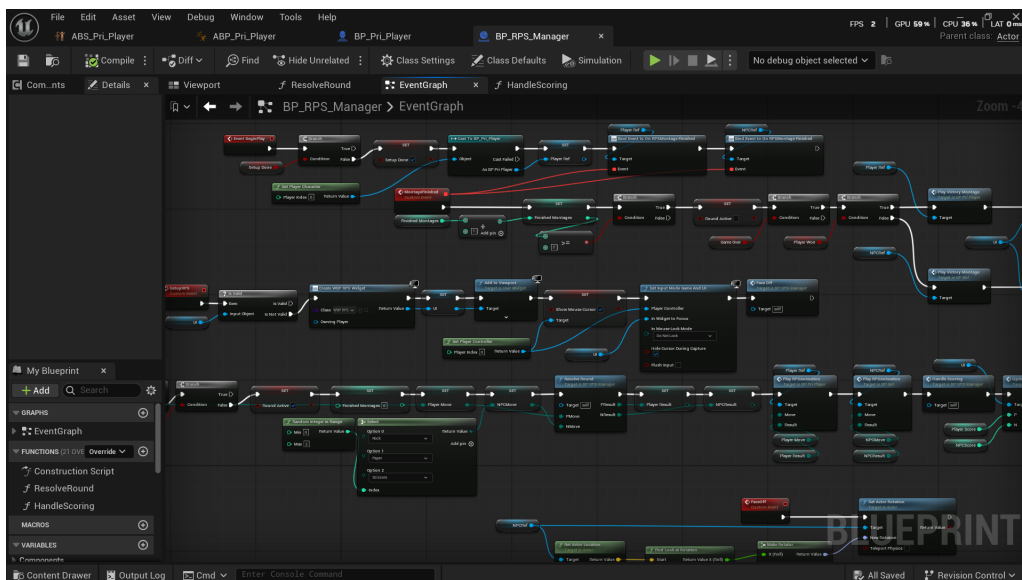
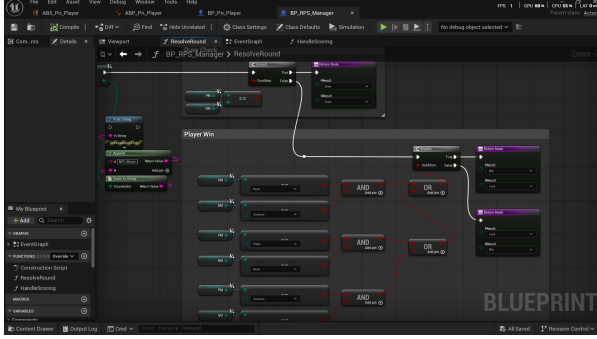


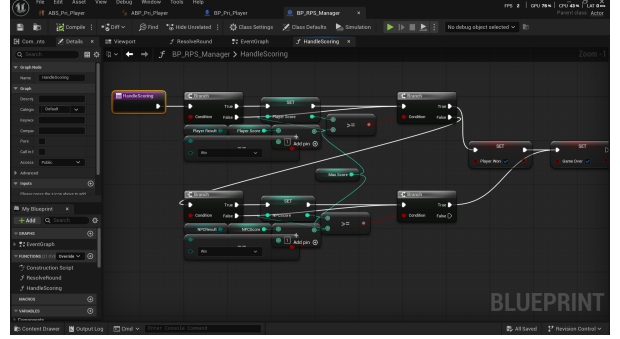
Figure 3: RPS manager Blueprint and overall system flow.

The project is split across three main blueprints: The player blueprint handles movement, input, animation playback, and the shift from normal locomotion into game mode when the interaction begins. The NPC blueprint handles its own animation response and facing behaviour. The manager handles the UI, move resolution, scores, and overall round flow.

Pressing the interaction key ('F') brings up the UI, the player picks rock, paper, or scissors, the NPC move is generated randomly, and the manager resolves the result before sending the chosen move and outcome to both characters. The UI also updates the score display and shows a result text after the game concludes.



(a) Move routing.



(b) Outcome and score flow.

Figure 4: Function-level Blueprint logic.

From the animation side, the system uses animation sequences through montages [3] and blend spaces [4]. The sequences provide the source clips, the montages turn them into controllable gameplay actions, and the blend spaces handle the more continuous transition between locomotion or idle and the ready state. The interaction itself is built as a staged sequence rather than one long clip, mainly because the source mocap was not clean enough to behave like one seamless performance.

The clip set used for the gameplay interaction was:

- 1 clip to enter the ready state,
- 1 clip that locks both players into the ready frame,
- 1 clip for the hand-tapping motion,
- 3 clips for the rock, paper, and scissors throws,
- 7 clips for the result phase, consisting of 3 win clips, 3 loss clips, and 1 draw clip,
- and 1 extra game-over animation per character, with a different dance for each.

This staged structure let me build the interaction around what the mocap could support instead of forcing everything into one long animation that would have been much harder to clean and synchronize. Most of these clips came from my own performance, which helped the player's retargeted animation sit more naturally on my avatar. The main exceptions

[illegible]

2.2 Blend Space and Animation Blueprint Setup

The player animation blueprint was built around a 2D blend space inside an Animation Blueprint setup [2, 4]. Along the X axis it used three locomotion clips: idle, walk, and a faster clip labelled as “fly”, which effectively takes the role of a run but with a more comedic

feel. These were controlled using the player’s movement velocity (‘Left Shift’ was used to adjust speed for running state). Along the vertical state axis, the ready clip is placed at state = 1, which allows the character to move from normal locomotion into readiness before the RPS montages begin. This was much cleaner than snapping directly from movement into a throw animation.

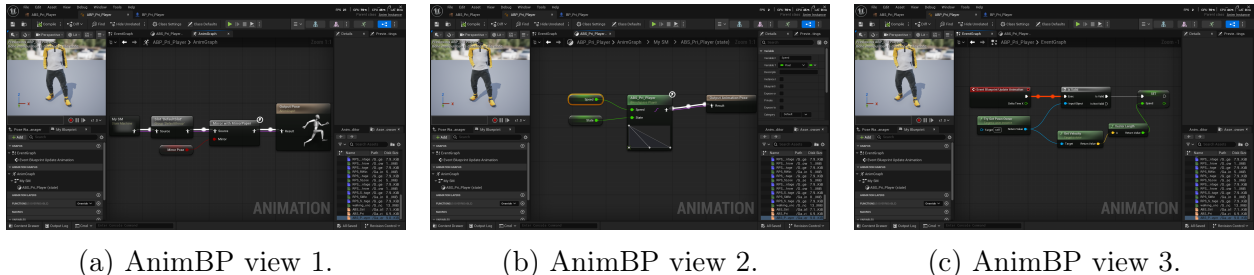


Figure 7: Player’s animation blueprint handling state and gameplay playback.

The NPC setup was simpler and used a 1D blend space that moved only between idle and ready, because they did not need the same locomotion complexity. It also maintained facing through a simple LookAt-style LERP rotation update toward the player, which kept the interaction from feeling disconnected. The NPC also received its own separately retargeted clip set, which helped avoid clipping issues that showed up more readily when trying to force the same retargeted material across both characters.

Smoothing was handled mainly through two decisions: the use of the ready-state blend-space transition before gameplay actions, and montage chaining through **On Blend Out** rather than **On Completed**. Using **On Completed** introduced a visible pause between stages, while chaining through blend-out gave the sequence a much more continuous feel. After testing different settings, I settled on a blend-in and blend-out time of roughly 0.5 seconds with a Hermite Cubic InOut curve, because that gave the smoothest result overall. It still did not remove every artifact, since some clips carried noticeable differences in pose, bone rotation, and body orientation from one stage to the next, but it made the interaction read much better than the harsher default transitions.

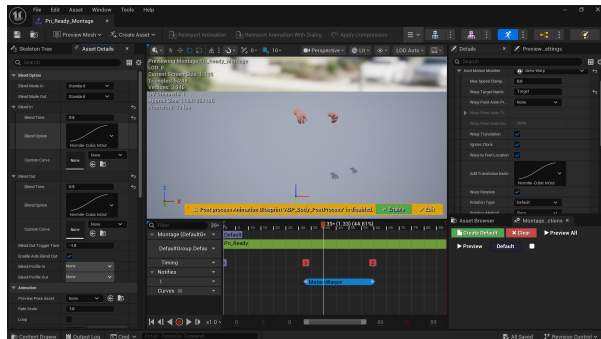
2.3 Keyframe Editing, Warping, and Synchronization

Even after the system structure was working, the source mocap still had obvious weaknesses. Some transitions contained weak poses, some timings were uneven, and some frames simply made the animation look worse when left untouched. Because of that, I used Sequencer to trim unwanted lead-in and tail frames, fix rotations where the poses broke down, and reduce visible jitter in the upper body and hands. This was one of the most useful motion-editing steps in the demo, because blending alone could only hide so much, especially movement.

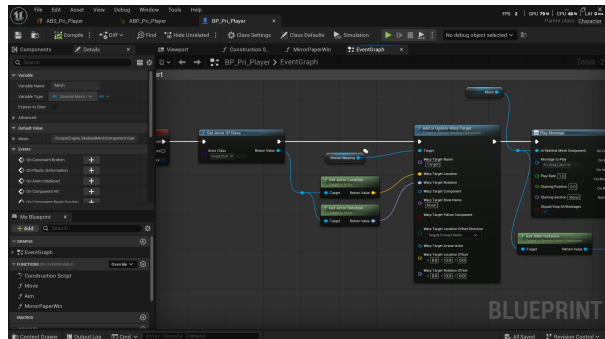
Mirroring was used only for the PaperWin animation to counteract the obviously wrong motion [9]. I did not apply it across the rest of the set because only this particular instance involved changing the throwing hand and caused a clear snap between transitions.

The last major problem was spatial alignment. Even when both characters were playing the correct animations at the correct time, the interaction still failed if their positions relative

to each other were wrong, so motion warping was used to align the player dynamically to the interaction target before or during montage playback [10]. To get it working properly, the montage had to play on the correct mesh, root motion extraction had to be configured correctly, and the warp target had to be updated at the right time.



(a) Warping setup.



(b) Alignment result.

Figure 8: Motion warping for spatial alignment.

Synchronization across both characters was still handled centrally by the manager blueprint. Both characters enter the ready phase, play the required montage stages, and report completion back to the manager before the next phase can continue. Without that, even small timing offsets were enough to make the exchange feel wrong.

2.4 Critical Evaluation

This demo works best when looked at as one animation system rather than a collection of separate tricks. Concatenation gave the interaction structure, smoothing stopped the stages from feeling too obviously stitched together, the blend-space setup gave the player a cleaner path into readiness, keyframe cleanup repaired weak source motion that blending could not hide, and motion warping fixed the alignment problem that remained even after the timing was working. If the montage staging is removed, the interaction loses structure. If smoothing is weak, the sequence feels broken apart. If the blend space is removed, the player snaps too harshly into the action. If keyframe cleanup is skipped, the source mocap issues become much more obvious. If warping is missing, the two characters never quite feel like they are sharing the same space properly due to poor orientation.

The main weakness still comes from the mocap source itself. Some pose mismatches remain visible, and no amount of blending can completely hide every inconsistency when the original clips are imperfect. There is also an unavoidable tradeoff between neatness and responsiveness, because longer blends make the motion read better but can also make the interaction feel slightly softer than intended. There is also a clear flaw in the design of motion warping working weaker, or at times not at all, in the NPC as an actor blueprint than in the case of a playable character, which was another motivation for my choice of designing one rather than simply having two actors just present in the scene, while also giving me a chance to present a more interesting animation blueprint and a 2D blend space.

3 Facial Performance Transfer

The third demo moves away from body motion and focuses instead on facial performance, using Unreal’s Live Link workflow and MetaHuman Performance assets [6, 7] together with clips from the RAVDESS dataset [12, 11] to generate facial animation and apply it to two MetaHumans: my own self-avatar and a female alternative character, and judging how well an automated performance pipeline works in practice.

I selected clips representing several emotional states, including happiness, sadness, anger, fear, and surprise. Keeping the same source actor across the chosen clips made the comparison cleaner, because the more interesting variation then came from the target MetaHuman rather than from switching performers. I chose RAVDESS over MEAD mainly for practical reasons: the RAVDESS download I used was about 500 MB for a pack, whereas MEAD was about 18 GB on average, so RAVDESS was much easier to work with due to storage limits.

3.1 Import and Processing Workflow

The source data came from the RAVDESS dataset [12, 11], where each filename encodes modality, vocal channel, emotion, intensity, statement, repetition, and actor identity, so for this demo I kept that structure as controlled as possible by using clips that all shared the audio-visual speech format, the same statement, the same actor, and the strong emotional intensity variant. The only thing I changed between clips was the emotion itself, which meant the comparison stayed controlled and focussed on differences in emotion and target-character interpretation rather than noise caused by mixing performers, delivery, or recording conditions. The clip set chosen was as follows:

- 01-01-03-02-02-01-13.mp4 for happy,
- 01-01-04-02-02-01-13.mp4 for sad,
- 01-01-05-02-02-01-13.mp4 for angry,
- 01-01-06-02-02-01-13.mp4 for fearful,
- and 01-01-08-02-02-01-13.mp4 for surprised.

One of the most useful findings in this demo was that the pipeline could be made much simpler by relying on Live Link Capture Manager [6] instead of overcomplicating the setup. Importing clips through this system automatically generated the associated capture data, media source, and audio assets, which reduced manual preparation considerably.

Each clip was then processed through a MetaHuman Performance asset [7] by assigning the imported capture data and a target MetaHuman. After processing, the resulting facial animation could be exported and used directly inside Sequencer. This made it possible to build the demo around a longer concatenated sequence of short emotional performances.

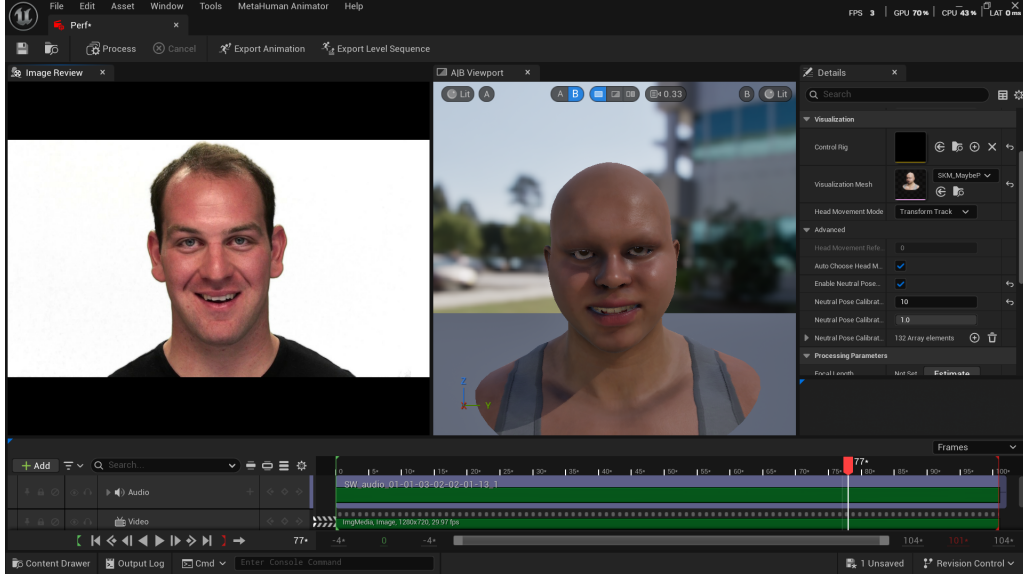


Figure 9: Facial performance transfer into the main MetaHuman with pose correction.

3.2 Application to Multiple MetaHumans

Applying the processed facial performance to more than one MetaHuman allowed me to compare how the same source expression reads across different facial structures and appearances. Although the underlying solve is the same in principle, the perceived result changes depending on the target face shape and proportions.

For subtle or moderate expressions, the transfer was generally convincing. For stronger expressions, some nuance could be lost or slightly exaggerated, and the quality became more dependent on the target character and the source video.

Looking at the source clips directly against the generated facial animation made that even clearer. Core facial behaviour such as mouth articulation, eye motion, and broad expression timing transferred consistently, and because all clips came from the same actor speaking the same line, the differences seen in the output could be read mainly as differences in the target MetaHuman rather than inconsistencies in the source data. The timing of the animation stayed stable across both characters, but the way the emotion was perceived still shifted depending on the face it was applied to. Anger and surprise, in particular, felt slightly stronger or harsher on some face shapes, while subtler expressions like fear lost a little fine detail. Lip motion was generally good, but still not exact enough to completely disappear as a solved process.

3.3 Critical Evaluation

Compared with a more manual facial pipeline, the combination of Live Link import and MetaHuman Performance processing gave a fairly direct path from source video to usable animation [6, 7]. That made it especially useful for quick experimentation and short emotional clips, where the aim was to compare performances rather than spend most of the time building setup around them.

At the same time, this is also the demo with the least direct editing control. Once the performance is solved, there is far less room to shape the result in the same hands-on way as in the body-animation demos, so the quality depends heavily on both the source video and the target face. That became especially obvious in the comparison between the two MetaHumans used here.

My own MetaHuman has a chubbier face with fuller cheeks and lips, and the eyes are also more almond-shape, so even though the solve remained usable, some expressions were softened or reduced when transferred and ended up feeling more adapted to my facial structure than directly identical to the source. I would place that translation at roughly 75% overall.

The alternative MetaHuman, by contrast, was a female character with a much sleeker face, and the expressions generally read more clearly on her despite the source clips coming from a male actor. On that face the translation felt stronger overall, and I would rate it closer to 90%.

Because of that, this demo feels less like manual motion editing and more like evaluation of the pipeline itself. Only by directly modifying the Face Rig of the respective MetaHumans and keyframing specific instances could bring about the missing realism and perfect adaption to resolve any artefacts. As such, the direct comparison between the source video, my self-avatar, and the alternative MetaHuman is the strongest part here.

References

- [1] Epic Games. *Animating MetaHumans with Control Rig in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/metahuman/animating-metahumans-with-control-rig-in-unreal-engine>.
- [2] Epic Games. *Animation Blueprints in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/unreal-engine/animation-blueprints-in-unreal-engine>.
- [3] Epic Games. *Animation Montage in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/unreal-engine/animation-montage-in-unreal-engine>.
- [4] Epic Games. *Blend Spaces in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/unreal-engine/blend-spaces-in-unreal-engine>.
- [5] Epic Games. *IK Rig Animation Retargeting in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/unreal-engine/ik-rig-animation-retargeting-in-unreal-engine>.
- [6] Epic Games. *Live Link in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/unreal-engine/live-link-in-unreal-engine>.
- [7] Epic Games. *MetaHuman Performance Asset*. URL: https://dev.epicgames.com/documentation/metahuman/metahuman-performance-asset?application_version=5.7.
- [8] Epic Games. *MetaHuman Techwear Outfit*. URL: <https://fab.com/s/189e540abe35>.
- [9] Epic Games. *Mirroring Animation in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/unreal-engine/mirroring-animation-in-unreal-engine>.
- [10] Epic Games. *Motion Warping in Unreal Engine*. URL: <https://dev.epicgames.com/documentation/unreal-engine/motion-warping-in-unreal-engine>.
- [11] Steven R. Livingstone and Frank A. Russo. *The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS)*. Zenodo dataset release. 2018. URL: <https://doi.org/10.5281/zenodo.1188976>.
- [12] Steven R. Livingstone and Frank A. Russo. “The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A Dynamic, Multimodal Set of Facial and Vocal Expressions in North American English”. In: *PLOS ONE* 13.5 (2018), e0196391. DOI: [10.1371/journal.pone.0196391](https://doi.org/10.1371/journal.pone.0196391). URL: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0196391>.
- [13] NOETA Marketstore. *Poor Man Outfit*. URL: <https://fab.com/s/f67dd250cbba>.
- [14] polycornStudio. *Casual Hoodie & Jeans Outfit for MetaHuman*. URL: <https://fab.com/s/59a0aa706a4f>.